

Implementing To-Do List APIs with Node.js, Express.js, and MongoDB

Objective:

Build the backend of a To-Do List app by implementing the identified APIs using Node.js, Express.js, and MongoDB.

Instructions:

In the previous assignments, you identified the necessary APIs for a To-Do List app. Now, it's time to implement these APIs using the Node.js runtime, Express.js framework, and MongoDB as the database. Follow the steps below to complete this assignment:

1. Set Up Your Project:
2. Configure MongoDB:
3. Implement API Logics:
4. Use Controller Service and Routes Structure.
5. Error Handling and Validation
6. Testing (Use Postman for testing):

Submission Guidelines:

- Push the entire codebase, including the main server file, route files, and any supporting modules On Github Repo.
- Provide a brief README.md explaining how to set up and run your To-Do List app.
- Include any necessary environment variables or configuration details in your README.
- Discuss any challenges you faced during the implementation and how you addressed them.

Submission Link : (Add Google form Link)

Note:

- Ensure that your code follows best practices for code organisation, readability, and maintainability.
- You can choose to use asynchronous programming and modular code where applicable.
- This assignment focuses on the practical implementation of APIs using Node.js, Express.js, and MongoDB.

Part 2:**Integrating To-Do List APIs with React Frontend****Objective:**

Integrate the previously built To-Do List APIs with the React frontend you created in previous assignments. Achieve seamless communication between the frontend and backend to make a fully functional To-Do List app.

Instructions:

In the previous assignments, you successfully built a React frontend for your To-Do List app and All Apis Required. Now, your task is to integrate the backend APIs into the frontend to create a fully functional application. Follow the steps below to complete this assignment:

1. Install Necessary Packages:

- Install any required packages for making HTTP requests from your React application (e.g. **Axios**).

2. Integrate API Calls:

- Modify your React components to integrate with the To-Do List APIs.

- Implement functions to make HTTP requests to the backend for:
 - Retrieving the list of tasks.
 - Creating a new task.
 - Updating task details.
 - Deleting a task.
 - Updating Status
 - Searching tasks etc.

3. Update UI Dynamically:

- Update the UI based on the data received from the backend.
- Ensure that tasks are displayed dynamically, and changes are reflected immediately.
- Implement loading indicators or error messages as needed.

4. User Interaction:

- Ensure that user interactions (e.g., adding a new task, marking a task as complete) trigger the corresponding API calls.
- Implement a responsive and user-friendly interface.

5. Error Handling:

- Handle errors gracefully on the frontend.
- Display appropriate error messages to users in case of API request failures.

6. Testing:

- Test your application thoroughly to ensure that API integration works as expected.
- Verify that data is synchronised between the frontend and backend.

7. Submission Guidelines:

Push the entire codebase, including React components and any new files you've created on the github.

Host The app on some hosting sites (for example use Render for backend and Netlify for frontend.)

Provide a brief README.md explaining how to set up and run your integrated To-Do List app.

Include any necessary environment variables or configuration details in your README.

Discuss any challenges you faced during the integration and how you addressed them.

Note:

- Ensure that your React code follows best practices for code organisation, readability, and maintainability.
- Consider implementing state management (e.g., using **useState** or a state management library) for handling application state.

Submission Guidelines:

- Deploy the Code on Netlify and provide the deployed link.
- Upload the Project on Github. Submit Github link also.
- Provide a brief overview of the decisions made during the enhancement process.